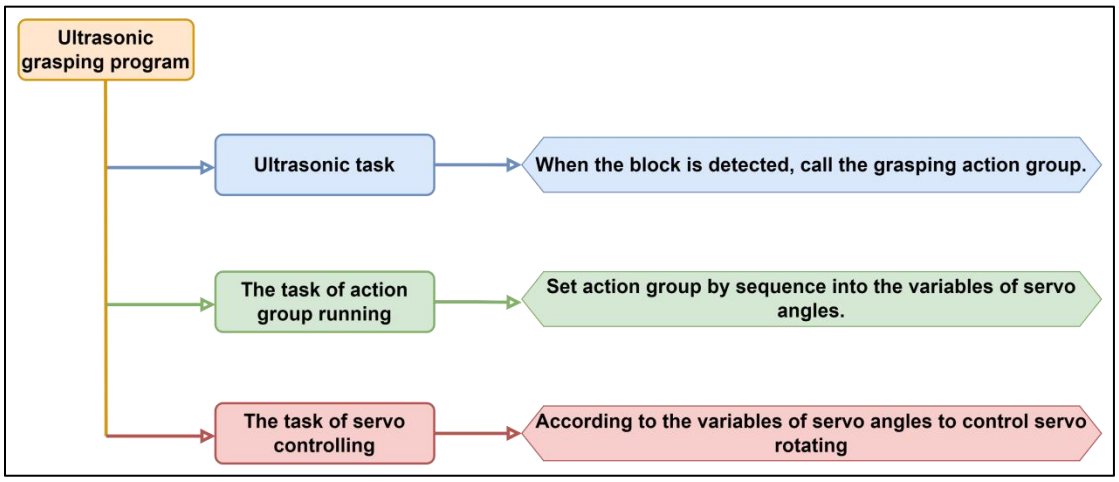
5.4 Ultrasonic Grasping

In this section, you can learn how to use a glowing ultrasonic module to detect the obstacle's distance, and program the robotic arm to grasp and release a block, as well as control the color of the RGB light.

1. Program Flowchart



2. Ultrasonic Sensor

This is a glowing ultrasonic ranging module. It adopts an I2C communication interface, which can read the range measured by an ultrasonic sensor through I2C communication.

2.1 Working Principle

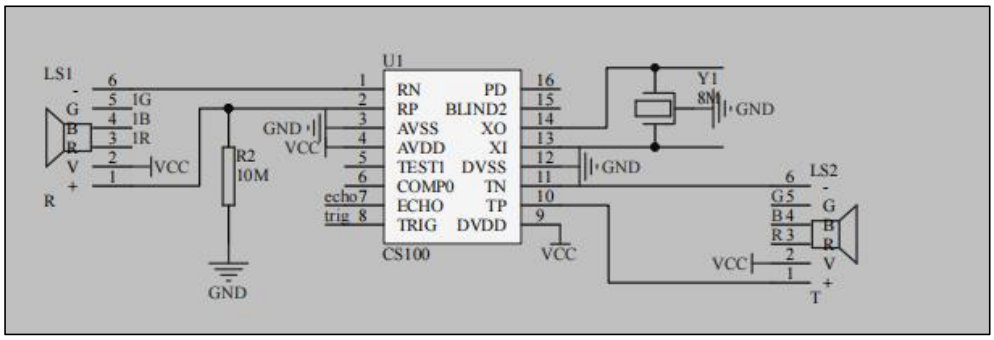
The ultrasonic sensor will automatically transmit 8 square waves at 40khz during ranging, and then detect whether there is a signal return. If there is a

signal return, it outputs a high voltage level, whose continuing time is the time of the ultrasonic from transmitting to returning.

Formula: measurement distance=(high voltage level time*sound speed (340m/s))/2.

2.2 Schematic Diagram

The sensor is controlled by the “CS100” chip. The chip’s pins (TP, TN) transmit 8 square waves at 40Khz, and the chip’s pins (RP, RN) detect echo signals. Then the distance value can be calculated by the formula “measurement distance=(high voltage level time*sound speed (340m/s))/2”.



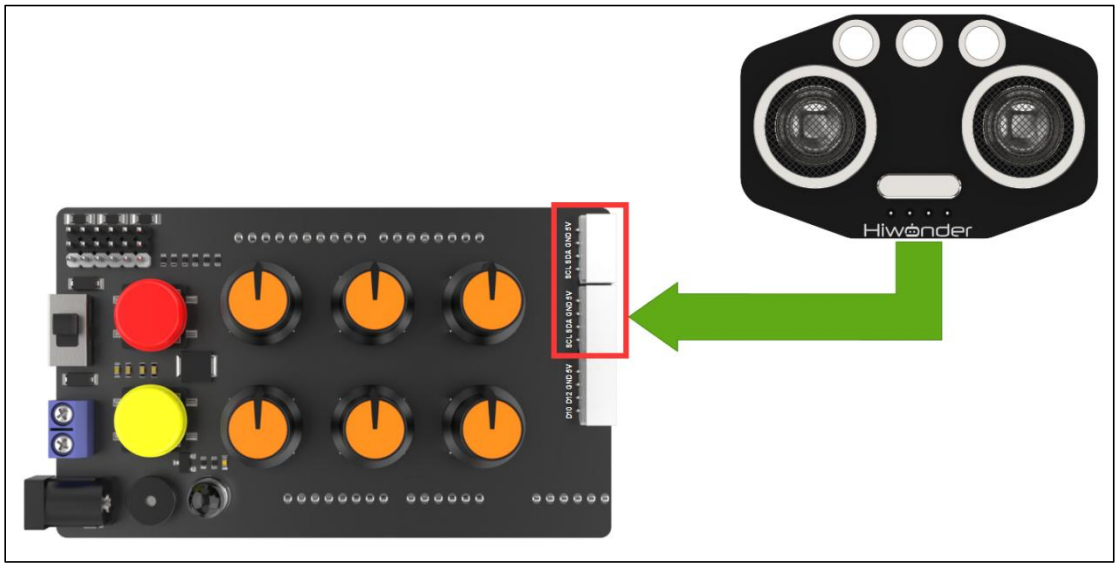
Supply Voltage :5V

Working Current: 2mA

Effective Ranging: 2cm - 400cm

2.3 Wiring

Connect the ultrasonic module to an I2C interface on the expansion board with the 4Pin wire.



3. Program Download

Note

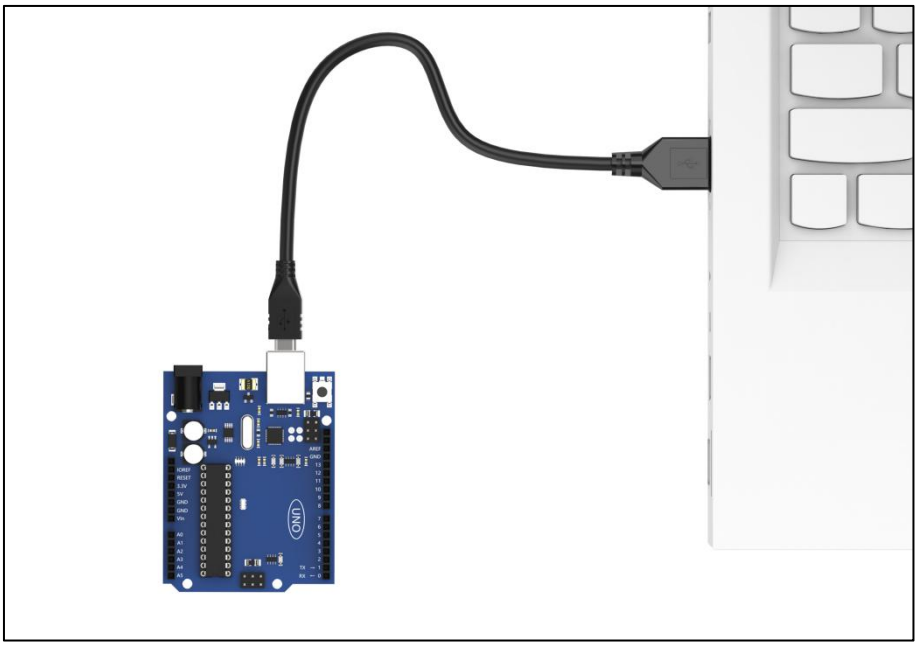
◆ Remove the Bluetooth module before downloading the program. Or the program will fail to download because of the serial port conflict.

◆ Please switch the battery box to the “OFF” when connecting the Type-B download cable. This action prevents the download cable from accidentally touching the power pins of the expansion board, which may cause a short circuit.

1) Locate the “MiniArm_Ultrasound_Grab.ino” program file in “Program File\MiniArm_Ultrasound_Grab” in the same directory of this section.

名称	修改日期
MiniArm_Ultrasound_Grab.ino	2024/2/27 14:21
tone.h	2024/2/27 14:21
Ultrasound.cpp	2024/2/27 14:21
Ultrasound.h	2024/2/27 14:21

2) Connect Arduino to the computer with the UNO cable (Type-B).



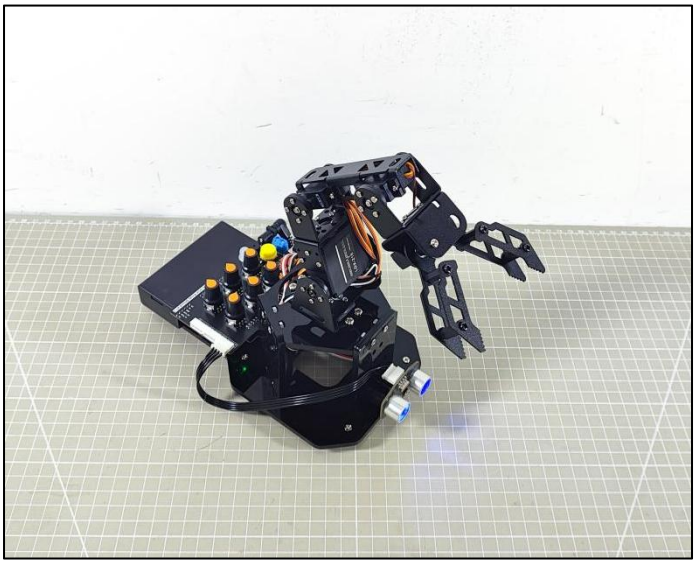
3) Click “Select Development Board”, and the software will automatically detect the current Arduino serial port. Next, click to connect.

4) Click to download the program into Arduino. Then just wait for it to complete.

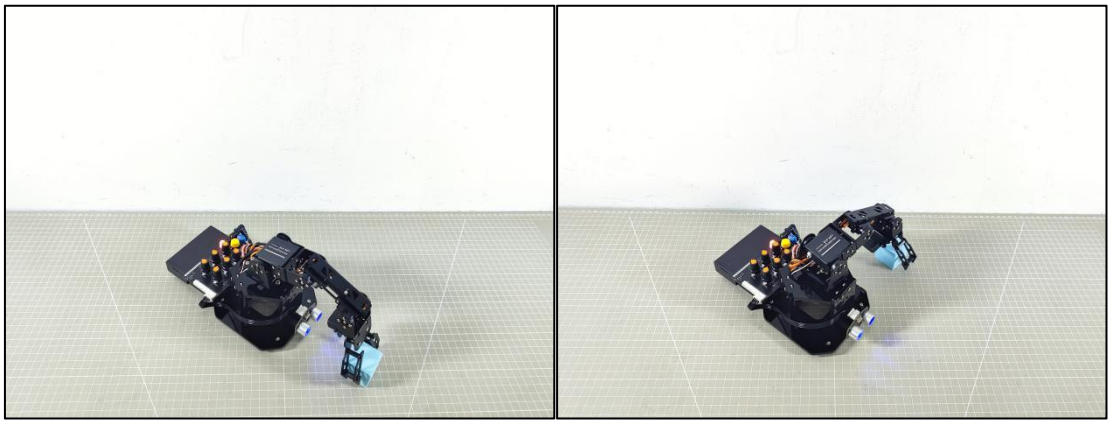
4. Program Outcome

After powered on, the robotic arm returns to the neutral position mode first. Then it gets into the preparatory action of grasping the block, and the RGB

light on the ultrasonic sensor simultaneously turns blue.



Place the block about 4cm in front of the ultrasonic sensor. The buzzer will make a short sound, then the robotic arm will grasp the block and place it on the left side (from the robotic arm's view). During the grasping process, the RGB light on the expansion board turns red.



5. Brief Program Analysis

5.1 Import Library File

```
#include <FastLED.h> //import LED library
#include <Servo.h> //import servo library
#include "tone.h" //tone library
#include "Ultrasound.h" //import ultrasonic library
#include <EEPROM.h>
```

Import the necessary libraries for the control of RGB, servo, and MiniArm, as well as the ultrasonic library for the game.

5.2 Pin Definition and Object Creation

1) Initially, the tone of two buzzers and the Arduino pins for hardware connections are defined, primarily including six servo pins, one buzzer pin, and one RGB LED pin.

```
const static uint16_t DOC5[] = { TONE_C5 };
const static uint16_t DOC6[] = { TONE_C6 };

/* pins definition */
const static uint8_t servoPins[6] = { 7, 6, 5, 4, 3, 2 };
const static uint8_t buzzerPin = 11;
const static uint8_t rgbPin = 13;
```

2) Then, create the control objects of RGB lights, action groups, and servos. The variables of the servo control are also created. The “limit_angle” sets angle limits for each joint of the robotic arm. The “servo_angles” array is used to store the actual angles of each servo, ranging from 0 to 180.

```
//servo control object
Servo servos[6];

//create ultrasonic object
Ultrasound ul;
 /* Angle values used in secondary development routines */
static uint8_t extended_func_angles[6] = { 73,10,161,57,90,90 };
/* Limitations on joint angles */
const uint8_t limt_angles[6][2] = {{0,82},{0,180},{0,180},{0,180},{0,180},{0,180}};
/* The angle value of the actual control of the servo */
static uint8_t servo_angles[6] = { 73,10,161,57,90,90 };
static int8_t servo_offset[5];
static uint8_t servo_expect[5];
static uint8_t eeprom_read_buf[16];
```

3) In the control object program of the action group “mini_servo.h”, the construction function defines the variable “extended_func_angles”, which is the desired angle value of servo angles. The related functions used to control executing action groups and read deviation are also defined.

```
class HW_ACTION_CTL{
  public:
  /* the using angles of secondary development example */
    uint8_t extended_func_angles[5] = { 41 ,12 ,174 ,68 ,84 };
    //Control execution action group
    void action_set(int num);
    int action_state_get(void);
    void action_task(void);
    void read_offset();
    int8_t* get_offset(void);

  private:
    //Action group control variables
    int action_num = 0;
    int8_t servo_offset[5];
    uint8_t eeprom_read_buf[16];
};
```

Regarding the desired and actual values, you can check the “servo_control” function called in the “loop” main function.

If you want the servo to gradually move towards the target position, you need to call “servo_angles[i] = servo_angles[i] * 0.9 + extended _func_ a ngles[i] * 0.1” in the control task function.

This allows the servo to move each time at 90% of the actual value and 10% of the expected value, so that the actual value will gradually approach the expected value. And when it is equal to the expected value, the robotic arm stops moving.

```
  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + action_ctl.extended_func_angles[i] * 0.1;
      if(servo_angles[i] < action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }else if(servo_angles[i] < action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + (action_ctl.extended_func_angles[i] * 0.1 + 1);
      if(servo_angles[i] > action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }
    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

4) Please note that task functions execute different control tasks. The “servo_control” function controls the servo. The “play_tune()” function sets the buzzer’s sound. The “tune_task” function controls the buzzer. And the “ultrasound_task” function reads the ultrasonic sensor data.

```
static void servo_control(void);  //Servo control
void play_tune(uint16_t *p, uint32_t beat, uint16_t len); /* Buzzer control interface */
void tune_task(void); /* Buzzer control task */
static void ultrasound_task(void);  // Ultrasonic task
```

5.3 Initialization Setting

1) The “setup ()” function mainly initializes associated hardware devices. Starting with the serial port, which sets the baud rate as “115200” and a read timeout as 500ms.

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  // set the read data timeout of serial port
  Serial.setTimeout(500);
```

2) Assign servo IO port and use the “FastLED” library to initialize the RGB light on the expansion board and connect it to the “rgbPin”. Set the color of RGB as blue by “rgbs[0] = CRGB(0, 0, 100)”. Next, use the “FastLED.show” function to display the set color.

```
// assign servo IO port
for (int i = 0; i < 6; ++i) {
  servos[i].attach(servoPins[i],500,2500);
}

//initialize the RGB light and control it
FastLED.addLeds<WS2812, rgbPin, GRB>(rgbs, 1);
rgbs[0] = CRGB(0, 0, 100);
FastLED.show();
```

3) After the deviation value of the servo is read, set the pin of the buzzer to output mode. Call the “tone()” function to make the buzzer sound. Then wait for 100ms before calling the “noTone()” function to stop the buzzer from sounding.

```
//Read deviation value
action_ctl.read_offset();

//Initialize the buzzer and it makes a sound
pinMode(buzzerPin, OUTPUT);
tone(buzzerPin, 1000);
delay(100);
noTone(buzzerPin);
ul.Color(0,0,255,0,0,255);
```

5.4 Ultrasonic Detection

After successful initialization, call the ultrasonic task function “ultrsound_task” in the “loop” main function, which is used to achieve the ranging function. Initially, it defines the variable “last_tick” for calculating the time intervals mainly.

```
// Ultrasonic task
void ultrasound_task(void)
{
  static uint32_t last_ul_tick = 0;
  static uint8_t step = 0;
  static uint8_t act_num = 0;
  static uint32_t delay_count = 0;

  // Interval 100ms
  if (millis() - last_ul_tick < 100) {
    return;
  }
  last_ul_tick = millis();

  // Obtain ultrasonic distance
  int dis = ul.Filter();
```

Then check if 100 milliseconds have passed since the last function “ultrasound_task” calling. If so, it will execute the following code. Or the function returns directly. The function “millis()” is the time since starting the Arduino board. In this way, the function ensures that the following distance detection tasks are performed every 100 milliseconds.

Next, the ultrasonic distance value after filtering is obtained by the “ul.Filte r” function, and the result will be stored in variable “dis”.

5.5 Execution Feedback

1) Detect the obtained obstacle’s distance value. If it ranges from 35mm to 50mm, the RGB lights will turn red and get into the next stage once the buzzer makes a sound. If the obstacle distance is not within the range of 35mm to 50mm during this time, the LEDs will not be lit up. The obtained distance value will be checked again.

```
switch(step)
{
  case 0:
    if(dis > 35 && dis < 50) //If the block is detected
    {
      rgbs[0].r = 250;
      rgbs[0].g = 0;
      rgbs[0].b = 0;
      FastLED.show();
      play_tune(DOC6, 300u, 1u);
      act_num = 1;
      step++;
    }else //If the block is not detected
    {
      rgbs[0].r = 0;
      rgbs[0].g = 0;
      rgbs[0].b = 0;
      FastLED.show();
    }
    break;
```

2) In “case 1”, delay is based on the value of “delay_count”. Each time “case 1” is entered, the ultrasonic task is called. Each call to the ultrasonic task takes approximately 100ms. Therefore, “case 1” first delays for 1000ms, then sets the action group number and executes the action group. After completion, it proceeds to the next stage.

```
case 1: //Wait for 1s to put the block in the specified place.
  delay_count++;
  if(delay_count > 10)
  {
    delay_count = 0;
    // Action group running
    action_ctl.action_set(act_num);
    act_num = 0;
    step++;
  }
  break;
```

3) “Case 2” signifies the waiting phase for action completion. After completing the recognition, waiting, and calling the action group, “Case 2”checks if the “action_num” variable is 0 to determine if the action group has

finished running. Once the action group has ended, the “step” stage variable will be reset to 0.

```
  case 2: //Wait for the action status to change to 0
    if(action_ctl.action_state_get() == 0)
    {
      step = 0;
    }
    break;
  default:
    step = 0;
    break;
}
```

5.6 Servo Control

1) In the servo control task, initially, use the “last_tick” variable to control the delay time.

```
// servo control task
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();
```

2) Then use the “action_ctl” object to obtain the desired value (extended_func_angles) of the servo. Combine it with the actual value (servo_angles) of the servo, and send the angle to be rotated to the servo to control servo rotating.

```
  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + action_ctl.extended_func_angles[i] * 0.1;
      if(servo_angles[i] < action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }else if(servo_angles[i] < action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + (action_ctl.extended_func_angles[i] * 0.1 + 1);
      if(servo_angles[i] > action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }
    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

Regarding the ideal expected value (the angle we want the servo to rotate to) and the actual expected value (the angle the servo can rotate to), the explanation is as below:

In the “action_ctl” object, “extended_func_angles” represents the ideal expected value, which excludes any deviation caused by the mechanical structure of the servo. However, in actual projects, the servo may have a certain range of deviation due to its mechanical structure. To control the servo angle more accurately, we calculate the angle that needs to be added to the ideal position to compensate for the deviation. This angle can be positive, negative, or zero. In actual servo control, the deviation value to the ideal expected angle will be added to obtain the actual expected angle.

6. Function Extension

How to modify the color gradient of an RGB LED from purple to yellow?

Please refer to the following steps:

1) Find mapping instructions in the program for controlling the RGB lights to change with the change of the distance. Set R and B in RGB elements changing as the change of “dis” by “map” function. “map (dis, 50,200,255,0)” maps the obstacle distance “dis” to the R element. And “map (dis, 50,200,0,255)” maps the obstacle distance “dis” to the B element.

Therefore, as the distance decreases, the proportion of red increases

while the proportion of blue decreases, resulting in a gradient color displayed as purple.

```
/* the color changes according to the variation. The closer it gets,
 the larger the R (red) and the smaller the B (blue). Conversely, the farther away, the opposite occurs */
int color[3] = {map(dis , 50 , 200 , 255 , 0),0,map(dis , 50 , 200 , 0 , 255)};
```

2) Change the B element in RGB to the G element. Note that the range of color elements is 0 to 255. Next, download the program again. The closer the obstacle is, the more the proportion of red and the less proportion of green. The RGB light displays yellow.

```
int color[3] = {map(dis , 50 , 200 , 255 , 0),map(dis , 50 , 200 , 0 , 255),0};
```

The glowing ultrasonic sensor is red

The color changes with the change of distance. The closer the distance, the bigger the R and the smaller the B. The further the distance, the opposite.

The closer the distance, the more curved of each finger; the further the distance, the straighter.

About the RGB color table, you can access : https://www.bchrt.com/tools/rgbcolor/ to research.

7. FAQ

Q:since the code was upload, the distance measured by the ultrasound is always 0.

A:please check that you have connected the 4-pin cable to the I2C interface and turn the knob on the expansion board to the initial position.

Q:the distance measured by the ultrasonic sensor is sometimes accurate,

and sometimes inaccurate.

A:please use a smooth and flat object for distance measurement.